

- A. `String.valueOf(3);`
 B. `String str = "hello";`
 C. `new String("hello");`
 D. `String[] strArray = new String[10];`
- 7、关于抽象类，下列叙述正确的是_____。
- A. 抽象方法必须包含于抽象类
 B. 抽象类的子类一定是具体类
 C. 抽象类不能定义构造方法
 D. 抽象类的父类一定是抽象类
- 8、如果在与 `class A` 不同包的 `class B` 中，语句 “`A a = new A(); A.i=10;`” 成立，则下列定义正确的是_____。
- A. `public class A { public inti = 2; }`
 B. `public class A { public static final inti = 3; }`
 C. `class A { public static inti = 4; }`
 D. `public class A { public static inti = 5; }`
- 9、下面关于继承的叙述正确的是_____。
- A. 子类实例函数能够覆盖父类实例函数，子类实例变量能够覆盖父类实例变量
 B. 子类实例函数能够隐藏父类实例函数，子类实例变量能够覆盖父类实例变量
 C. 子类实例函数能够覆盖父类实例函数，子类实例变量能够隐藏父类实例变量
 D. 子类实例函数能够隐藏父类实例函数，子类实例变量能够隐藏父类实例变量
- 10、若 `String s1=new String("CS1"); String s2 = new String("CS2"); boolean b1 = (s1==s2); boolean b2 = (s1.equals(s2));` 则一下对应 `b1`、`b2` 的值正确的为_____。
- A. true, false B. false, true C. true, true D. false, false

得分	评卷人

二、填空（每空 2 分，共 20 分）

- Java 中除了基本数据类型的变量为值类型外，其它类型的变量都是 引用 类型。
- 若一个构造方法不调用本类的其它构造方法，则其实际执行的第 1 条语句调用 基类构造方法。
- 在接口中定义 `double` 类型的成员变量，则这个变量的默认修饰符是 `public static final`。
- 创建 `char[][]a=new char[3][4]` 类型数组后，`a[1]` 元素的类型为 `char[]`。
- 分支 `if` 语句的 `else` 部分总是和 最近 的 `if` 匹配。
- `Integer` 装箱方法的参数个数只能是 1 个参数。
- java 的泛型是通过 擦除 法实现的。
- 循环 `for(int i = 0; i < 30; i += 5) {i++;}` 一共会执行 5 次循环。

9.异常的 finally 部分 总是 会被执行。

10.布尔表达式!(x!=2)^(x==2)的运算结果是 false。

得分	评卷人

三、改错题（每小题 10 分，共 20 分）

1、指出下面程序 javaOne.java 的错误之处及其原因。

```
packagemyCode;
publicclassjavaOne {
    public static int main(String [] str) { //(1)返回类型应为void
        int h=null; //(2)初始化值不能是引用类型的null
        int s=h=0;
        do {
            s+=h++;
        };while (h<10); //(3)循环体只能是一条语句，分号又是一条语句
        JOptionPane.showMessageDialog(null, "result="+s); //(4)没使用import引入
        System.exit(1.0); // (5) 必须是整型
    }
}
```

2、指出下面程序 javaTwo.java 的错误之处及其原因。

```
package myProgram;
import java.util.Stack;
interface myItfc{ void f( ); }
public class javaTwo implementsmyItfc{
    public void f( ) { javaTwo.main(null);}
    public static void main(String [] str) {
        myItfc m=new myItfc( ); // (1) 接口没有构造方法，且不能产生实例对象
        myItfc h=new javaTwo( );
        h.f( ); // (2) 这个调用导致无穷递归
        Stack<Integer> []s=new Stack<Integer>[20]; // (3) 不能使用<Integer>
        Stack<Integer>t=s[1].clone(); // (4) 没有公共的clone函数
        s[1].push('A'); // (5) 'A'不是int或Integer类型
    }
}
```

得分	评卷人

四、程序填空题（每小题 5 分，共 25 分）

1、写出下面程序的运行结果。（第 3 个结果 3 分，其它结果各 1 分）

```
package javaApplication;

public class TestPassArraySimple{

    public static void main(String[] args) {

        int x =1;
```

```

        int[] y = new int[10];
        int[] z = new int[10];
        y[0] = 15; z[0] = 30;
        m(x,y,z.clone());
        System.out.println("x is " + x);           //(1)
        System.out.println("y[0] is " + y[0]);      //(2)
        System.out.println("z[0] is " + z[0]);      //(3)
    }

    public static void m(int x, int[] y, int[] z) { x = 101; y[0] = 301; z[0] = 501; }
}

```

输出结果为:

(1) x is 1

(2) y[0] is 301

(3) z[0] is 30

2、写出下面程序的运行结果。(每个结果 1 分)

```

package javaApplication;

public class Circle
{
    private double radius;
    private static int count = 0;
    public Circle(double r){ radius = r; count++; }
    public Circle(){ this(1.0); }
    protected void finalize() throws Throwable{ count--; super.finalize(); }
    public static void main(String[] args)
    {
        Circle c1 = new Circle();
        Circle c2 = new Circle(15.0);
        System.out.println("count="+ count); //(1)
        Circle c3 = c2;
        System.out.println("count="+ count); //(2)
        c1=c2;
        System.out.println("count="+ count); //(3)
        Circle[] c4 = new Circle[10];
        System.out.println("count="+ count); //(4)
        System.gc();
        System.out.println("count="+ count); //(5)
    }
}

```


}

输出结果为:

(1) count=2

(2) count=2

(3) count=2

(4) count=2

(5) count=1

3、写出下面程序运行结果。(第3个结果3分,其它结果各1分)

```
package javaApplication;
```

```
class Person {
```

```
    public Person() {System.out.println("Person's constructor"); }
```

```
}
```

```
class Employee extends Person {
```

```
    public Employee() {System.out.println("Employee's constructor");}
```

```
}
```

```
class Faculty extends Employee {
```

```
    public Faculty() {System.out.println("Faculty's constructor"); }
```

```
    public Faculty(String s) {System.out.println("Faculty's constructor with: " + s); }
```

```
}
```

```
public class ConstructorDemo {
```

```
    public static void main(String[] args) {
```

```
        new Employee ( ); // (1)
```

```
        new Faculty ( ); // (2)
```

```
        new Faculty("hello"); // (3)
```

```
    }
```

```
}
```

输出结果为:

(1) Person's constructor

Employee's constructor

(2) Person's constructor

Employee's constructor

Faculty's constructor

(3) Person's constructor

Employee's constructor

Faculty's constructor with: hello

4、请写出下面程序运行结果。(第3个结果3分,其它结果各1分)

```

package javaApplication;

abstract class A {public abstract void g();}

class B extends A { public void g() { System.out.println("Greeting from B"); } }

class C extends B { public void g() { System.out.println("Greeting from C"); } }

class D extends C { public void g() { System.out.print("Greeting from D "); super.g(); } }

public class Test{

    static void sendGreeting(A a) { a. g(); }

    public static void main(String[] args){

        sendGreeting (new B()); //(1)

        sendGreeting (new C()); //(2)

        sendGreeting (new D()); //(3)

    }

}

```

输出结果为:

- (1) Greeting from B
- (2) Greeting from C
- (3) Greeting from DGreeting from C

5、请写出下面程序运行结果。(每个结果 1 分)

```

package javaApplication;

class A {
    public static int x=0;
    public int y=2, z=3;
    public A(int y, int z){
        x++;
        this.y+=y;
        this.z+=z;
    }
}

class B extends A {
    public static int x=4;
    public int y=5, z=6;
    public B(int y, int z){
        super(y, z);
        x++;
        this.y+=y;
    }
}

```



```

        this.z+=z;
    }
}
public class JavaApplication {
    public static void main(String[] args) {
        A a=new A(3, 5);
        A b=new B(7, 9);
        System.out.println("a.y="+a.y);           //(1)
        System.out.println("a.z="+a.z);           //(2)
        System.out.println("b.y="+b.y);           //(3)
        System.out.println("b.z="+b.z);           //(4)
        System.out.println("a.x+b.x="+a.x+b.x);   //(5)
    }
}

```

输出结果为:

- (1) a.y=5
- (2) a.z=8
- (3) b.y=9
- (4) b.z=12
- (5) a.x+b.x=4

得分	评卷人

五、编程题(15 分)

编程实现以下矩形类 Rectangle、圆类 Circle 和 Mygeo 类中的 main 方法。假设类要实现的几何图形的接口 Geometry 定义如下:

```

interface Geometry{
    double pi=3.1416;
    double area(); //计算面积
    double perimeter(); //计算周长
}

```

请定义一个矩形类 Rectangle 和一个圆类 Circle, 并实现接口 Geometry 声明的方法。矩形类 Rectangle 存有矩形的宽 width 和高 height, 而圆类 Circle 只存圆的半径 radius。假定程序名为 Mygeo.java, 而程序运行的命令行格式可能如下:

- (1) java Mygeorectangle 100 20
- (2) java Mygeocircle 20

其中 (1) 指明了矩形的宽度 100 和高度 20, (2) 指明了圆的半径 20。主方法 main 需要处理命令行参数, 计算给定矩形 (或圆) 的面积和周长, 并调用 System 类的方法输出计算结果。

```

package javaApplication;
interface Geometry{
    double pi=3.1416;
    double area(); //计算面积
    double perimeter(); //计算周长
}

```

```

    }
    class Rectangle implements Geometry{
        double width, height;
        public Rectangle(double width, double height){
            this.width=width;
            this.height=height;
        }
        @Override    public double    area( ){ return width*height;}
        @Override    public double    perimeter( ){ return 2*(width+height);}
    }
    class Circle implements Geometry{
        double radius;
        public Circle(double radius){ this.radius=radius;}
        @Override    public double    area( ){ return Geometry.pi*radius*radius;}
        @Override    public double    perimeter( ){ return 2*Geometry.pi*radius;}
    }
    public class Mygeo {
        public static void main(String[] args){
            Geometry g=null;
            if (args.length<1) {
                System.out.println("Usage: Java  Mygeo  rectangle  100  20");
                System.out.println("        Java  Mygeo  circle  20");
                return;
            }
            if(args[0].equals("rectangle") && args.length==3) //或只判断参数个数
                g=new Rectangle(Double.parseDouble(args[1]),Double.parseDouble(args[2]));
            else if(args[0].equals("circle")&&args.length==2) //或只判断参数个数
                g=new Circle(Double.parseDouble(args[1]));
            else return;
            System.out.println(args[0]+"s area="+g.area( ), perimeter="+g.perimeter( ));
        }
    }
}

```